

HDOC Day-24

NAME: Yug Nirwal
SAPID: 590033354
SUBJECT: C programming
DATE: 17-09-26

Algorithm

1. Start the program.
2. Display a menu containing the five increasing patterns and Exit option.
3. Read the user choice.
4. If the choice is from 1 to 5, read the number of rows n.
5. Use a switch statement to select the required pattern.
6. Use nested loops to print the selected pattern. For the consecutive-number pattern, maintain a counter starting from 1.
7. For the alphabet pattern, print characters from A onward and allow rows from 1 to 26.
8. If the choice is invalid, display an error message.
9. Stop.

Test-Case Table

Choice	n	Pattern	Expected Output	Status
1	1	Right-angled triangle	*	Pass
1	4	Right-angled triangle	* / * * / * * * / * * * *	Pass
2	1	Row-number triangle	1	Pass
2	4	Row-number triangle	1 / 2 2 / 3 3 3 / 4 4 4 4	Pass
3	1	Natural-number pattern	1	Pass
3	4	Natural-number pattern	1 / 1 2 / 1 2 3 / 1 2 3 4	Pass
4	1	Consecutive-number triangle	1	Pass

4	4	Consecutive-number triangle	1 / 2 3 / 4 5 6 / 7 8 9 10	Pass
5	1	Alphabet pattern	A	Pass
5	4	Alphabet pattern	A / A B / A B C / A B C D	Pass

C Program

```
#include <stdio.h>
```

```
void starPattern(int n) {
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
}

void rowNumberPattern(int n) {
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) printf("%d ", i);
        printf("\n");
    }
}

void naturalNumberPattern(int n) {
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) printf("%d ", j);
        printf("\n");
    }
}

void consecutivePattern(int n) {
    int i, j, num = 1;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) printf("%d ", num++);
        printf("\n");
    }
}

void alphabetPattern(int n) {
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) printf("%c ", 'A' + j - 1);
        printf("\n");
    }
}

int main() {
    int choice, n;
    printf("INCREASING PATTERNS\n");
    printf("1. Increasing right-angled triangle\n");
    printf("2. Increasing row-number triangle\n");
    printf("3. Increasing natural-number pattern\n");
    printf("4. Consecutive-number triangle\n");
    printf("5. Increasing alphabet pattern\n");
    printf("6. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    if (choice == 6) return 0;
    if (choice < 1 || choice > 5) {
```

```

    printf("Invalid choice\n");
    return 0;
}

printf("Enter number of rows: ");
scanf("%d", &n);
if (n <= 0) {
    printf("Number of rows must be positive\n");
    return 0;
}
if (choice == 5 && n > 26) {
    printf("For alphabet pattern, enter rows from 1 to 26\n");
    return 0;
}

switch (choice) {
    case 1: starPattern(n); break;
    case 2: rowNumberPattern(n); break;
    case 3: naturalNumberPattern(n); break;
    case 4: consecutivePattern(n); break;
    case 5: alphabetPattern(n); break;
}
return 0;
}

```

Compilation and Execution

Save the program as hdoc24.c. Compile it using: gcc hdoc24.c -o hdoc24

Run it using: ./hdoc24

The following terminal-style images show the program tested against every prepared test case.

Test-Case Execution Images

Test Case 1: Choice 1, n = 1 - Right-angled triangle

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

```
Enter your choice: 1
```

```
Enter number of rows: 1
```

```
*
```

Test Case 2: Choice 1, n = 4 - Right-angled triangle

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 1

Enter number of rows: 4

```
*  
* *  
* * *  
* * * *
```

Test Case 3: Choice 2, n = 1 - Row-number triangle

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 2

Enter number of rows: 1

```
1
```

Test Case 4: Choice 2, n = 4 - Row-number triangle

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 2

Enter number of rows: 4

1

2 2

3 3 3

4 4 4 4

Test Case 5: Choice 3, n = 1 - Natural-number pattern

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 3

Enter number of rows: 1

1

Test Case 6: Choice 3, n = 4 - Natural-number pattern

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 3

Enter number of rows: 4

1

1 2

1 2 3

1 2 3 4

Test Case 7: Choice 4, n = 1 - Consecutive-number triangle

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 4

Enter number of rows: 1

1

Test Case 8: Choice 4, n = 4 - Consecutive-number triangle

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 4

Enter number of rows: 4

1

2 3

4 5 6

7 8 9 10

Test Case 9: Choice 5, n = 1 - Alphabet pattern

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 5

Enter number of rows: 1

A

Test Case 10: Choice 5, n = 4 - Alphabet pattern

```
(kali@kali) - [~/Desktop/cprog]
```

```
$ ./a.out
```

INCREASING PATTERNS

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 5

Enter number of rows: 4

A

A B

A B C

A B C D

Result

The menu-driven C program was compiled and executed successfully. All five increasing patterns were produced correctly for the prepared test cases using a switch statement and nested loops without arrays.